

Prácticas de PSyD

Daniel Báscones García

21 de julio de 2026

Índice

1. Práctica 0: Configuración del entorno de programación	2
1.1. Paso 1: Configuración para máquina virtual	2
1.2. Paso 2: Creación de la Máquina Virtual	2
1.3. Paso 3: Configuración del ToolKit de STM	3
1.4. Paso 4: Configuración de las herramientas de Zephyr	3
2. Práctica 1: Periféricos mapeados en memoria (MMIO): Entrada/Salida de propósito general (GPIO)	5
2.1. Objetivos	5
2.2. Fundamentos teóricos	5
2.3. Desarrollo de la práctica	9
2.3.1. Creación del proyecto	9
2.3.2. Depuración del proyecto	9
2.3.3. Desarrollo del proyecto	12
2.3.4. Solución	15

1. Práctica 0: Configuración del entorno de programación

Las prácticas de la asignatura de PSyD se desarrollan en un entorno ya preparado dentro de una máquina virtual. Esto es debido a la comodidad y portabilidad que ofrece, ya que con trasladar un simple fichero, cualquiera puede tener todas las herramientas disponibles en su ordenador. No obstante, también se ofrece aquí una guía de instalación para quien quiera instalarlo todo de manera nativa en su ordenador. La guía se ha realizado para Ubuntu. Su adaptación a otros sistemas operativos (MacOS, o Windows con WSL) es relativamente directa, al estar las herramientas disponibles para todas las plataformas.

1.1. Paso 1: Configuración para máquina virtual

Sigue esta parte sólo si quieres utilizar la máquina virtual proporcionada con todo el kit de herramientas, o si quieres crear una propia desde cero. Si quieres hacerlo de manera nativa, salta a Apartado 1.3.

- Asegúrate de tener instalado **VirtualBox** en su versión más reciente. Puedes descargarlo desde <https://www.virtualbox.org/wiki/Downloads>.
- Importante también descargar, desde el mismo enlace, el “VirtualBox Extension Pack”. Sin él, perderemos funcionalidades de la MV, entre otros, la capacidad de conectarle USBs (como el depurador ST-Link de la placa).

1.2. Paso 2: Creación de la Máquina Virtual

Crea una nueva máquina virtual e instala **Ubuntu** (recomendado versión **24.04 LTS**, descargable desde <https://ubuntu.com/download>). Configura el hardware asignándole los siguientes recursos recomendados:

- **Memoria RAM:** 8192 MB
- **Procesadores:** 4 núcleos (Cores).
- **Memoria de Vídeo:** 128 MB.
- **Tamaño del disco:** 60GB.
- **Tipo de instalación:** Manual. No utilizar la automática ya que instala cosas de más.

Una vez seleccionados los recursos, lanza la máquina y el instalador comenzará automáticamente. Deberás seleccionar cuestiones como el nombre de la máquina, usuario, contraseña... elige el que más te guste. El resto de la configuración se puede dejar por defecto. Ten en cuenta que el proceso de instalación puede tardar una una media hora (dependiendo del sistema anfitrión).

Una vez arranque la máquina, es importante instalar los extras de invitado (“Guest additions”), para conseguir que anfitrión y host se comunique adecuadamente (y puedan por ejemplo compartir portapapeles, ajustar el tamaño de pantalla, etc). Para ello haz click en **devices > insert guest additions CD image**. Esto cargará un disco en la MV (panel izquierdo). Sitúa la terminal en el directorio del CD (botón derecho sobre el explorador de archivos y click en “abrir terminal aquí”). Deberás ejecutar una serie de comandos:

```
1 # Actualizar paquetes existentes
2 > sudo apt update && sudo apt upgrade
3 # Instalar herramientas para compilar las guest additions
4 > sudo apt install bzip2 build-essential dkms linux-headers-$(uname -r)
5 # Instalar guest additions
6 > sudo ./VBoxLinuxAdditions.run
```

1.3. Paso 3: Configuración del ToolKit de STM

Independientemente de si quieres utilizar tu propio sistema operativo para instalar las herramientas, o instalarlas en la máquina virtual, se hace de la misma manera. A continuación se muestran todas las herramientas, repositorios, etc necesarios para tener todo lo necesario en la asignatura. Si prefieres no realizar la instalación, puedes descargar la máquina virtual directamente configurada desde [ENLACE](#).

- Instalar STM32CubeIDE (la herramienta principal de desarrollo) desde aquí: <https://www.st.com/en/development-tools/stm32cubeide.html>
- Instalar STM32CubeMX (para configuración avanzada) desde aquí: <https://www.st.com/en/development-tools/stm32cubemx.html>
- Instalar STM32CubeProgrammer (para compatibilidad con zephyr) desde aquí: <https://www.st.com/en/development-tools/stm32cubeprog.html>
- Instalar picocom para comunicar con la placa con posterioridad:

```
1 > sudo apt-get install picocom
```

1.4. Paso 4: Configuración de las herramientas de Zephyr

Para poder utilizar las herramientas de Zephyr, y compilar el sistema operativo en tiempo real para las placas, hay que hacer una serie de configuraciones relativamente complejas, que incluyen la descarga de varios repositorios. Aquí están las instrucciones oficiales, que replicamos en este documento: https://docs.zephyrproject.org/latest/develop/getting_started/index.html#getting-started.

```
1 # Instala las dependencias necesarias para trabajar con zephyr:
2 > sudo apt install --no-install-recommends git cmake ninja-build \
3     gperf ccache dfu-util device-tree-compiler wget python3-dev \
4     python3-venv python3-tk xz-utils file make gcc gcc-multilib \
5     g++-multilib libsdl2-dev libmagic1
6 # Crea una carpeta para el paquete de zephyr:
7 > mkdir ~/zephyrproject
8 # Crea un entorno virtual de python
9 > python3 -m venv ~/zephyrproject/.venv
10 # Cambia al entorno virtual para activar los paquetes de python
    necesarios
11 > source ~/zephyrproject/.venv/bin/activate
12 # Instala West (gestor de dependencias) e inicializa el proyecto
13 > pip install west
14 > west init -m https://github.com/zephyrproject-rtos/zephyr ~/
    zephyrproject
15 > cd ~/zephyrproject
16 > west update
17 # Exporta zephyr para que sea encontrado por otras herramientas
18 > west zephyr-export
19 # Instala las dependencias:
20 > west packages pip --install
21 # Instala el SDK de zephyr:
22 > cd ~/zephyrproject/zephyr
23 > west sdk install #(Usar primero --help para poner solo la toolchain de
    st)
```

Ya estamos listos para probar zephyr con un ejemplo y ver que funciona:

```
1 # Copy the blinky sample to a new folder
2 > cp -r zephyr/samples/basic/blinky my_blinky
3 > cd my_blinky
4 > west build -b stm32f412g_disco
5 (otra terminal) > picocom /dev/ttyACM0 -b 115200 -d 7 -p o
6 > west flash
```

2. Práctica 1: Periféricos mapeados en memoria (MMIO): Entrada/Salida de propósito general (GPIO)

2.1. Objetivos

- Comprender el concepto de **Memory-Mapped I/O (MMIO)**.
- Analizar la organización del mapa de memoria del microcontrolador **STM32F723IE**.
- Aprender a manipular directamente los registros de entrada/salida mediante máscaras de bits en C.

2.2. Fundamentos teóricos

En un procesador de propósito general, numerosas funciones se realizan a través de *periféricos*, componentes anexos al procesador con funciones específicas.

En la arquitectura ARM, los periféricos no se controlan con instrucciones dedicadas, sino que se comunican con la CPU a través del bus del sistema (*AXI-AHB Bus Matrix*) asignándoles direcciones físicas dentro del mapa de memoria de 32 bits (4GB).

Cada periférico posee un conjunto de **registros de control, estado y datos** ubicados en direcciones consecutivas a partir de una dirección base (*Base Address*). Escribiendo y leyendo en esas direcciones, conseguimos comunicarnos con los periféricos, que a su vez se conectarán con los diferentes pines del procesador. Los pines servirán para su conexión con el exterior con otros elementos, como LEDs, botones, pantallas, puertos, etc.

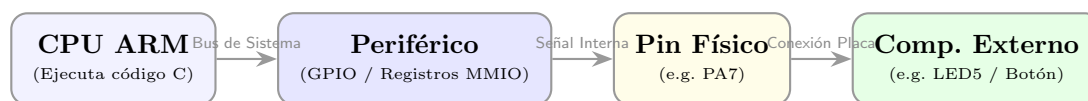


Figura 1: Flujo de control desde el código en CPU hasta el componente exterior a través de MMIO.

En nuestro caso, disponemos de un procesador **STM32F723IE**. Podemos consultar el manual de referencia [rm0431](#) para ver una lista completa de sus periféricos y las direcciones de memoria donde se ubican sus registros. La sección [rm0431 1.6](#) detalla todo este mapa de memoria, que se muestra en las Figuras 2 y 3

De aquí ya podemos deducir las direcciones de los diferentes controladores de GPIO. Vemos que están nombrados con letras (desde GPIOA hasta GPIOI). Por ejemplo, el controlador de GPIOA está en la dirección `0x40020000`. ¿Pero claro, cómo lo manejamos? Para ello debemos conocer todos los registros que se encuentran en esa zona de memoria. Esa información la tenemos en [rm0431 6.4](#).

A modo de ejemplo, se muestra en la Figura 4 la información referente al registro **MODER** del controlador de GPIO. Observamos que, en una palabra de 32 bits, contiene la configuración de modo de pin (Input/Output/Función alternativa/Analógico) para hasta 16 pines diferentes. Es decir, utiliza 2 bits para la configuración de cada pin. Los dos bits menos significativos corresponden al pin 0, y los 2 más significativos al pin 15, con el resto entre medias. Escribiendo en este registro configuraremos el modo de cada uno de estos pines.

Hemos visto pues que podemos configurar ciertos puertos de entrada/salida, que a su vez cuentan con numerosos pines. Pero, ¿en qué puerto/pin se sitúa, por ejemplo, un LED?. Para ello, debemos acudir a la documentación de la placa, donde se han diseñado físicamente las conexiones desde el procesador, hasta los diferentes elementos de soporte. En concreto, en la sección [um2140 5.16](#) tenemos la lista de todas las conexiones de botones y LEDs. Lo vemos en la Figura 5

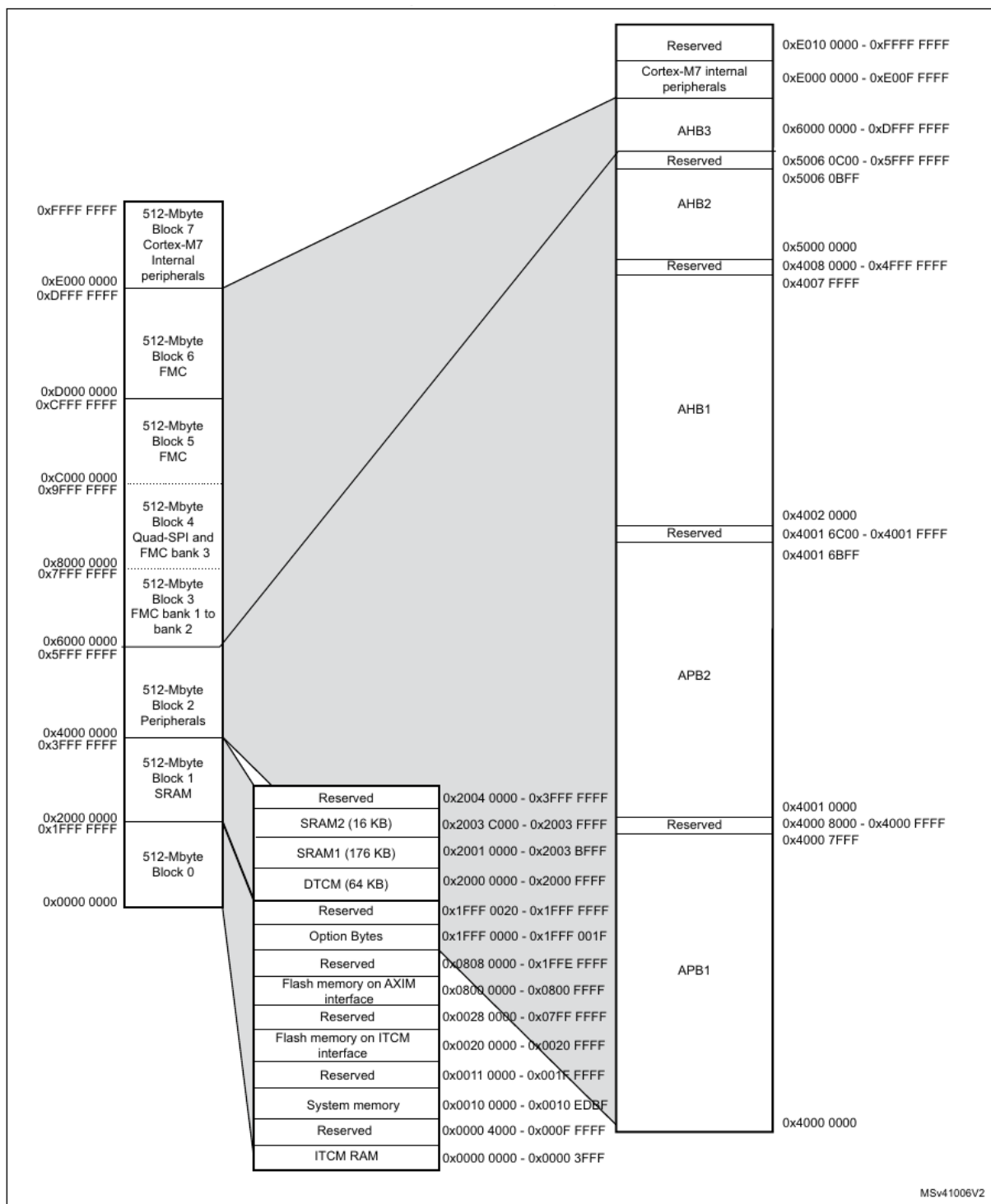


Figura 2: Mapa de memoria del **STM32F723IE**

Table 1. STM32F72xxx and STM32F73xxx register boundary addresses

Boundary address	Peripheral	Bus	Register map
0xA000 1000 - 0xA0001FFF	QuadSPI Control Register	AHB3	Section 13.5.14: QUADSPI register map on page 378
0xA000 0000 - 0xA000 0FFF	FMC control register		Section 12.8.6: FMC register map on page 349
0x5006 0800 - 0x5006 0BFF	RNG	AHB2	Section 16.7.4: RNG register map on page 458
0x5006 0000 - 0x5006 03FF	AES		Section 17.7.18: AES register map on page 510
0x5000 0000 - 0x5003 FFFF	USB OTG FS		Section 32.15.64: OTG_FS/OTG_HS register map on page 1288
0x4004 0000 - 0x4007 FFFF	USB OTG HS	AHB1	Section 32.15.64: OTG_FS/OTG_HS register map on page 1288
0x4002 6400 - 0x4002 67FF	DMA2		Section 8.5.11: DMA register map on page 252
0x4002 6000 - 0x4002 63FF	DMA1		
0x4002 4000 - 0x4002 4FFF	BKPSRAM		Section 5.3.27: RCC register map on page 192
0x4002 3C00 - 0x4002 3FFF	Flash interface register		Section 3.7.9: Flash interface register map
0x4002 3800 - 0x4002 3BFF	RCC		Section 5.3.27: RCC register map on page 192
0x4002 3000 - 0x4002 33FF	CRC		Section 11.4.6: CRC register map on page 275
0x4002 2000 - 0x4002 23FF	GPIOI		Section 6.4.11: GPIO register map on page 212
0x4002 1C00 - 0x4002 1FFF	GPIOH		
0x4002 1800 - 0x4002 1BFF	GPIOG		
0x4002 1400 - 0x4002 17FF	GPIOF		
0x4002 1000 - 0x4002 13FF	GPIOE		
0x4002 0C00 - 0x4002 0FFF	GPIOD		
0x4002 0800 - 0x4002 0BFF	GPIOC		
0x4002 0400 - 0x4002 07FF	GPIOB		
0x4002 0000 - 0x4002 03FF	GPIOA		

Figura 3: Detalle del mapa de memoria del **STM32F723IE**

6.4.1 GPIO port mode register (GPIOx_MODER) (x = A to K)

Address offset: 0x00

Reset value: 0xA800 0000 for port A

Reset value: 0x0000 0280 for port B

Reset value: 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODER[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Figura 4: Detalle del registro MODER

Table 5. Control port assignment

Reference	Color	Name	Comment
B1	BLUE	USER	Alternate function Wake-up
B2	BLACK	RESET	-
LD1	BLUE	ARDUINO	PA5
LD2	RED	5 V Power	-
LD3	RED	Fault Power	Current upper than 625 mA
LD4	RED/GREEN	ST-LINK COM	Green during communication
LD5	RED	USER1	PA7
LD6	GREEN	USER2	PB1
LD7	RED	USB OTG HS OVCR	PH10
LD8	GREEN	V _{BUS} USB HS	PB13
LD9	RED	USB OTG FS OVCR	PB10
LD10	GREEN	V _{BUS} USB FS	PA9

Figura 5: Pines para LEDS y botones de la placa **32F723EDISCOVERY**

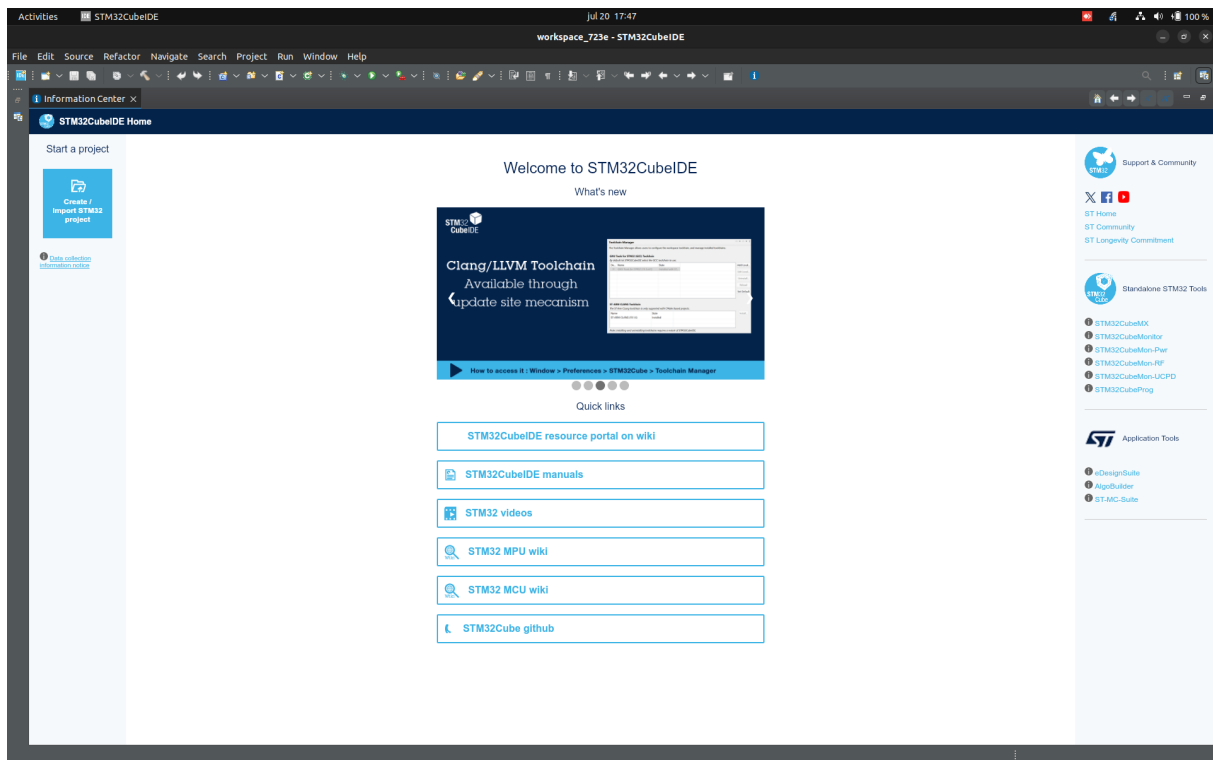


Figura 6: Ventana inicial de la aplicación

Finalmente, sabemos que, por ejemplo, el LED5 se controla desde el puerto GPIOA, en el PIN 7. El puerto A está en la dirección `0x40020000`, y dispone de varios registros que deberemos configurar adecuadamente para controlar, finalmente, el LED.

2.3. Desarrollo de la práctica

2.3.1. Creación del proyecto

Vamos a controlar, directamente y escribiendo en registros del procesador, un LED (en concreto, el LED5 que acabamos de ver). Pero antes, debemos crear un proyecto en STM32CubeIDE, la herramienta de desarrollo de ST. Para ello:

1. Abrimos STM32CubeIDE, buscándolo en el menú del sistema. Elegimos un workspace (o carpeta de trabajo) donde trabajar. Tras abrirlo, veremos el programa de la Figura 6
2. Creamos un nuevo proyecto con **File > STM32 Project Create / Import**.
3. Seleccionamos **STM32CubeIDE Empty Project** (Figura 7)
4. En **Board Selector**, buscar 723e y seleccionar la única placa disponible (Figura 8).
5. Finalmente, dar un nombre al proyecto y terminar (Figura 9).

2.3.2. Depuración del proyecto

Ahora deberíamos tener un proyecto en blanco para trabajar (Figura 10). Antes de nada, vamos a probar que la conexión con la placa funciona, y podemos compilar y depurar el proyecto. Para ello:

1. Haz doble click en la carpeta **Src** dentro de tu proyecto, y luego, en **main.c**.

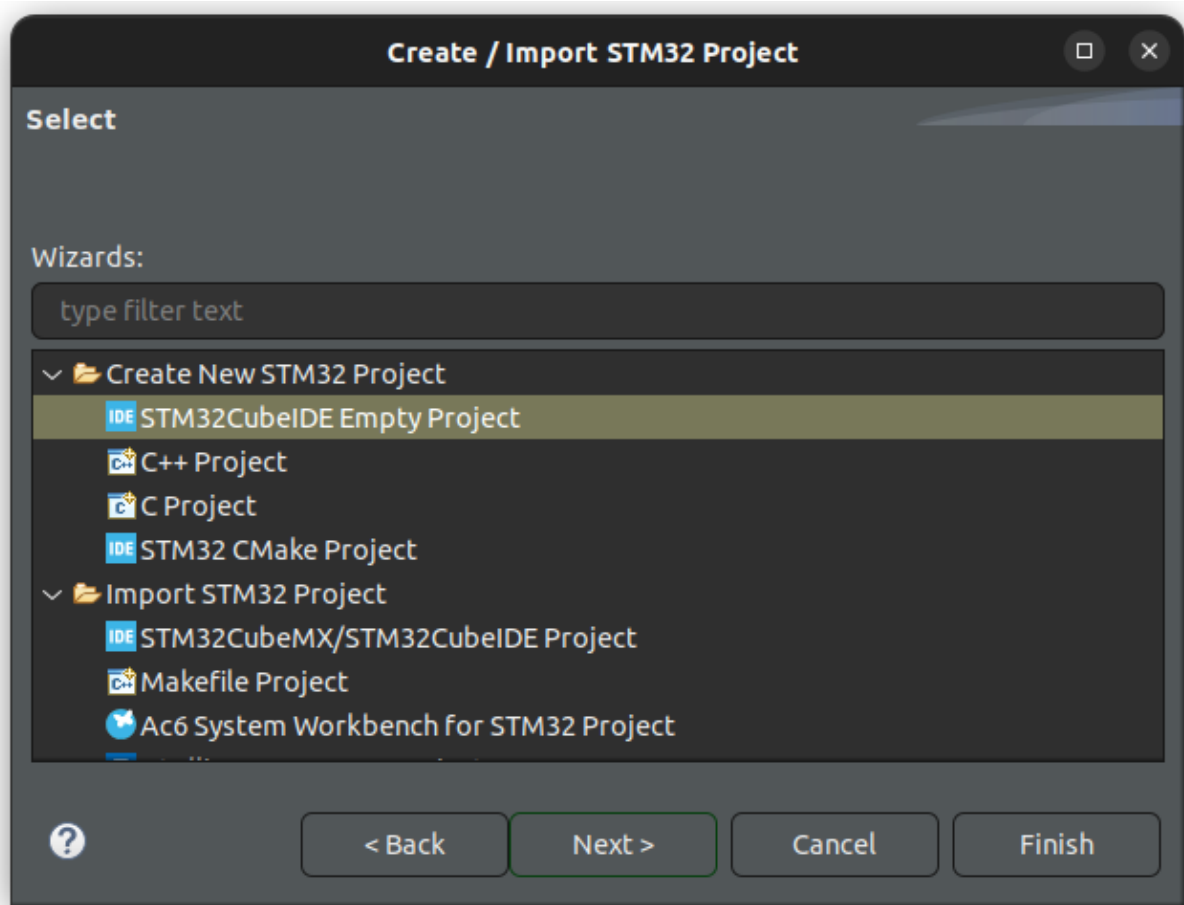


Figura 7: Creación de proyecto

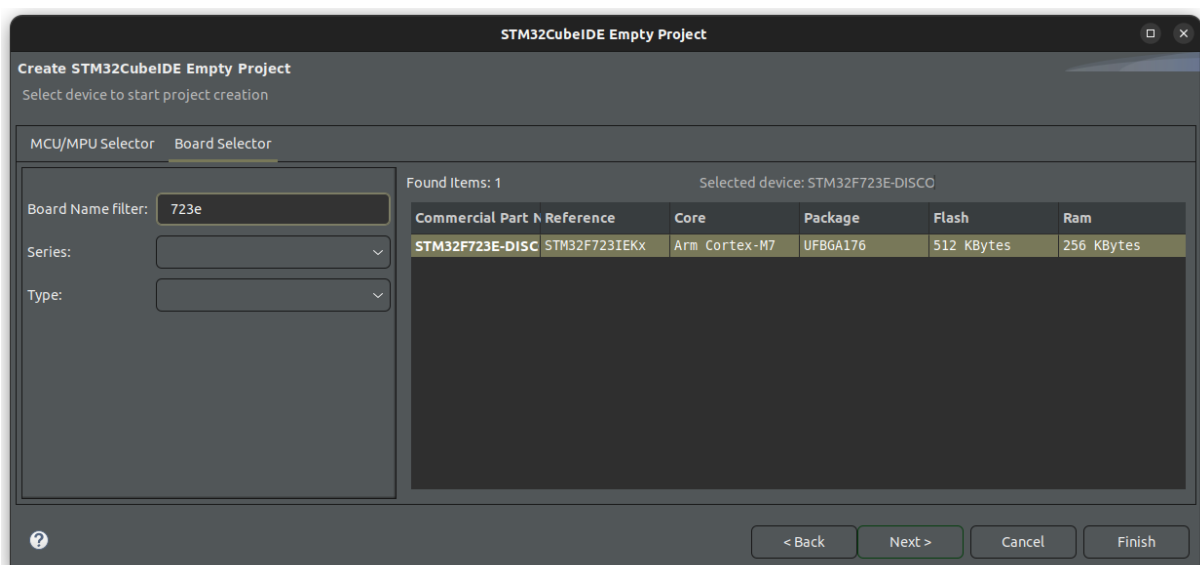


Figura 8: Selección de placa

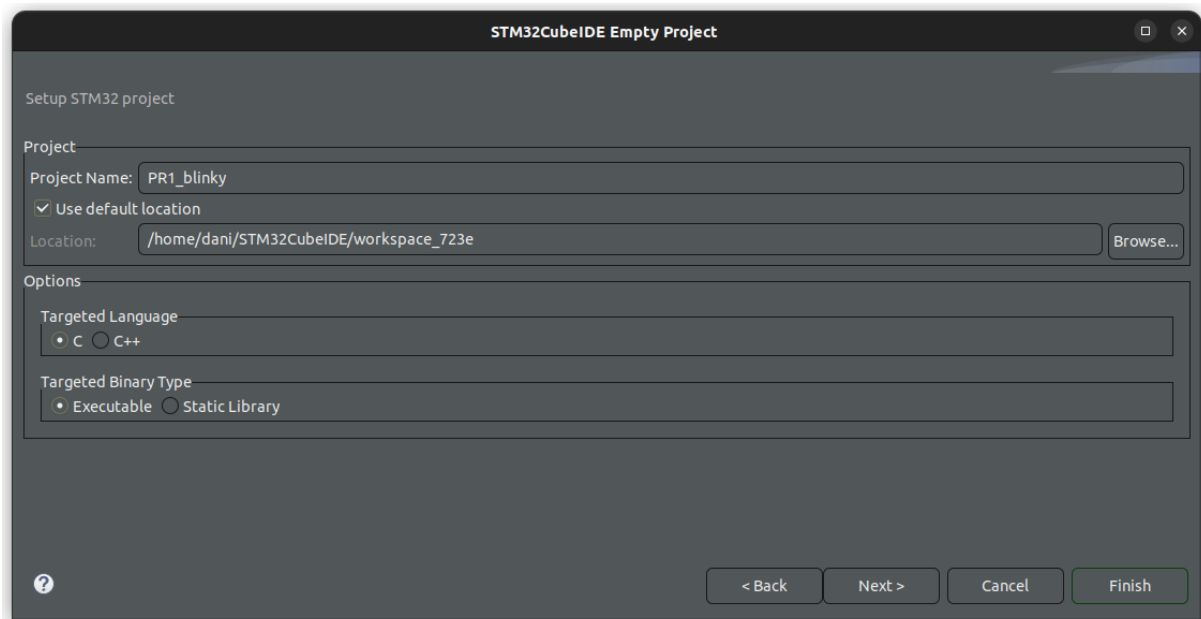


Figura 9: Selección de nombre de proyecto

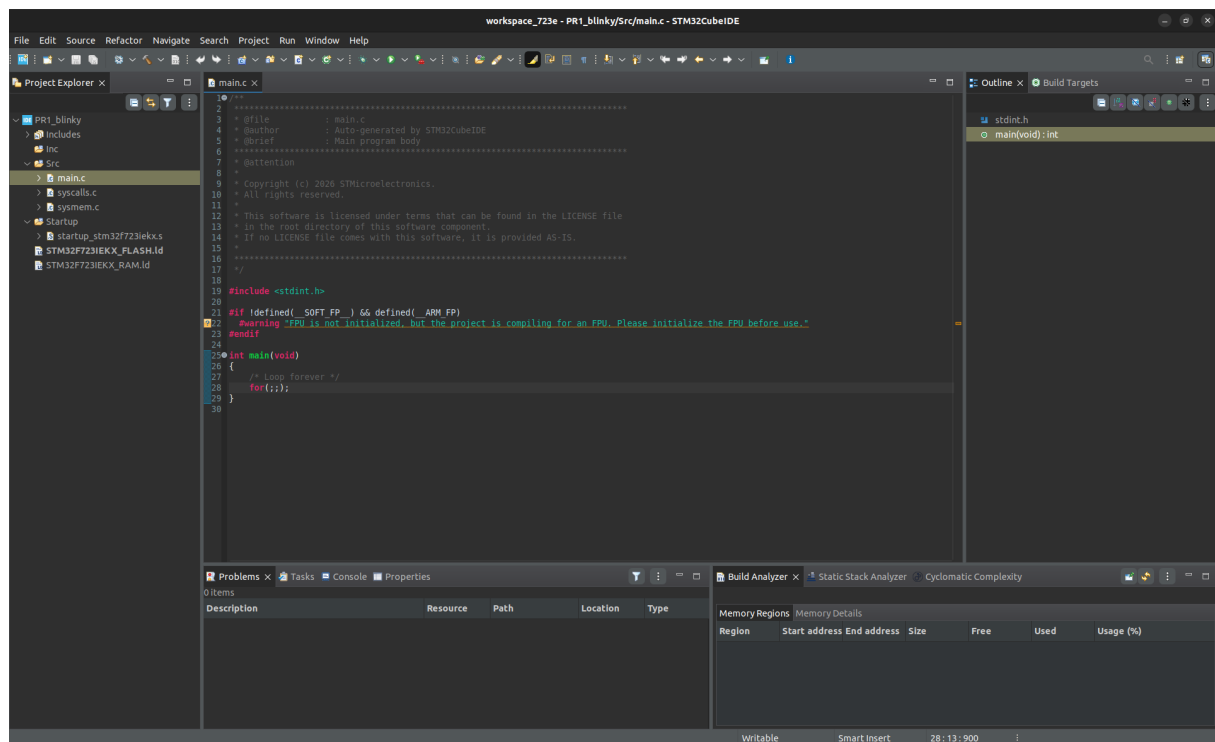


Figura 10: Aplicación por defecto

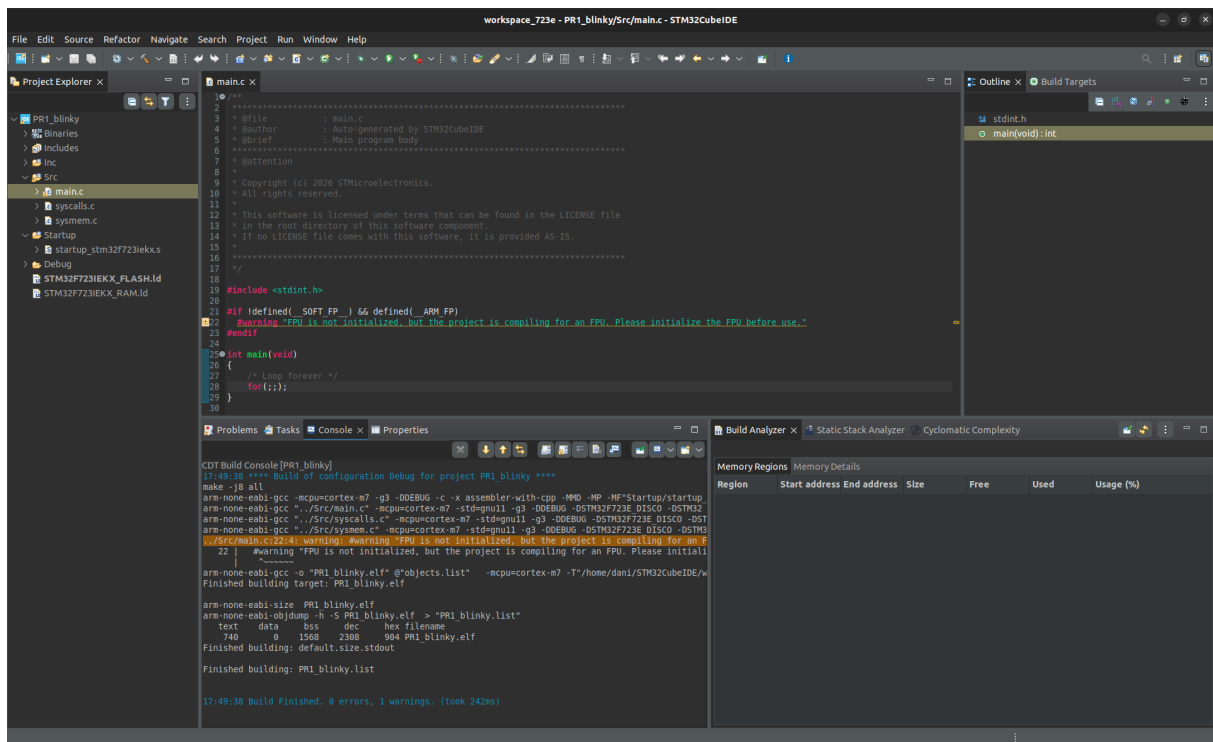


Figura 11: Log tras construir la aplicación

2. Con el archivo abierto, haz click en el martillo (build) que se encuentra en la barra superior de tareas. Verás el log de compilación en la terminal (Figura 11).
3. Finalmente, y si no ha habido problemas, haz click en el icono de depuración (debug) en la misma barra superior (el bichito verde).
4. Acepta la configuración de depuración por defecto (Figura 12) y vuelve a aceptar cuando pida cambiar a la vista de depuración.

Ahora puedes depurar el proyecto poco a poco. Entre otras, el depurador permite:

- Ir paso a paso por las líneas de código.
- Colocar *breakpoints* para parar la ejecución al llegar a ciertas líneas.
- Inspeccionar el contenido de los registros del procesador.
- Inspeccionar el contenido de los registros de los diferentes periféricos.
- Inspeccionar el contenido de la memoria.

Se recomienda explorar todas estas funcionalidades, que se pueden ver en la Figura 13.

2.3.3. Desarrollo del proyecto

Finalmente, debemos programar nuestro código para encender el LED. Para ello, habrá que escribir en las direcciones concretas de los registros de GPIO, en este caso de GPIOA-PIN7. Recuerda que la documentación de los registros disponibles está en [rm0431 6.4](#).

- Configurar GPIOA-PIN7 como salida en **MODER**.
- Poner el pin en tipo *push-pull* en **OTYPER**.

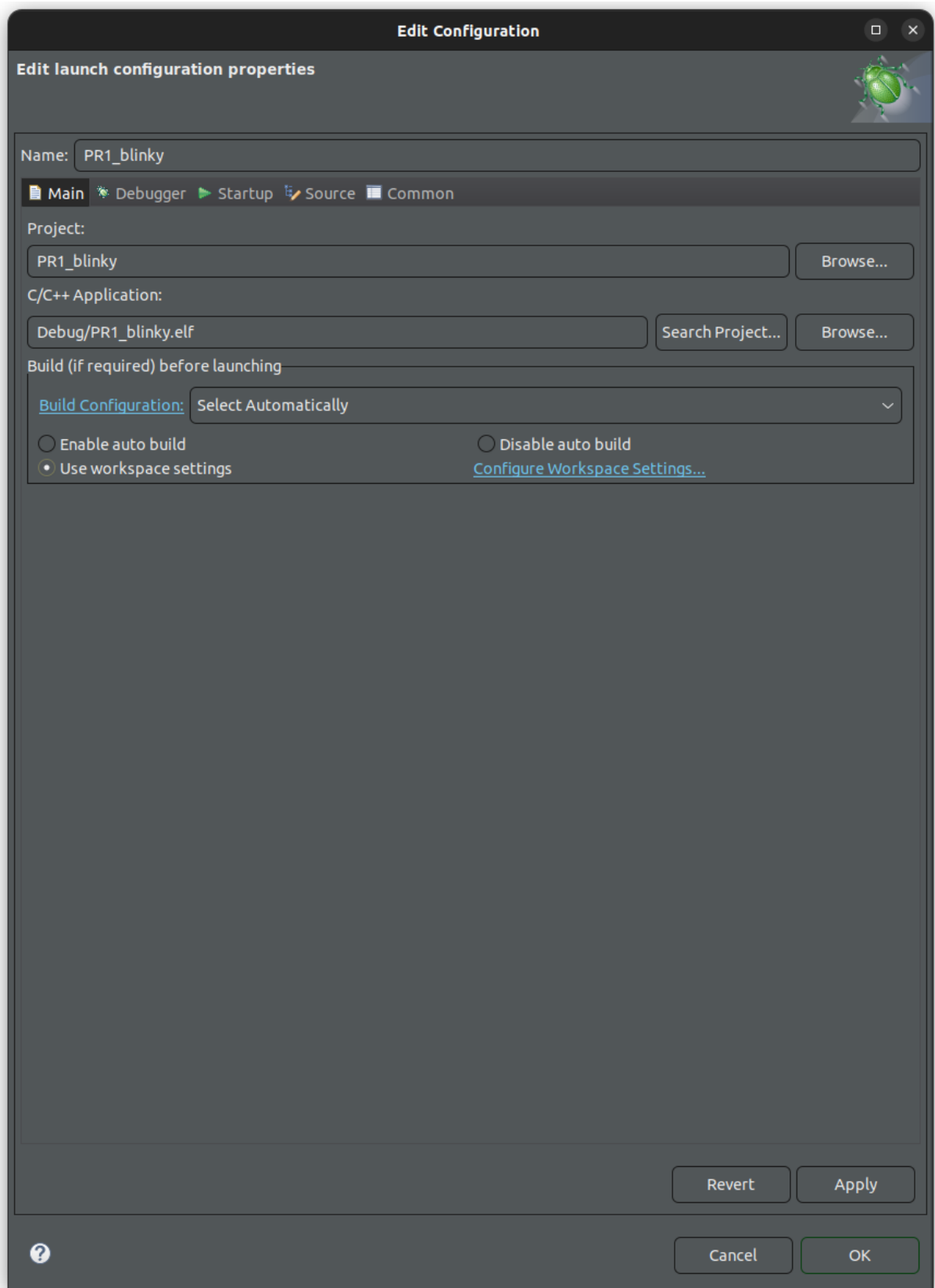


Figura 12: Selección de depuración por defecto

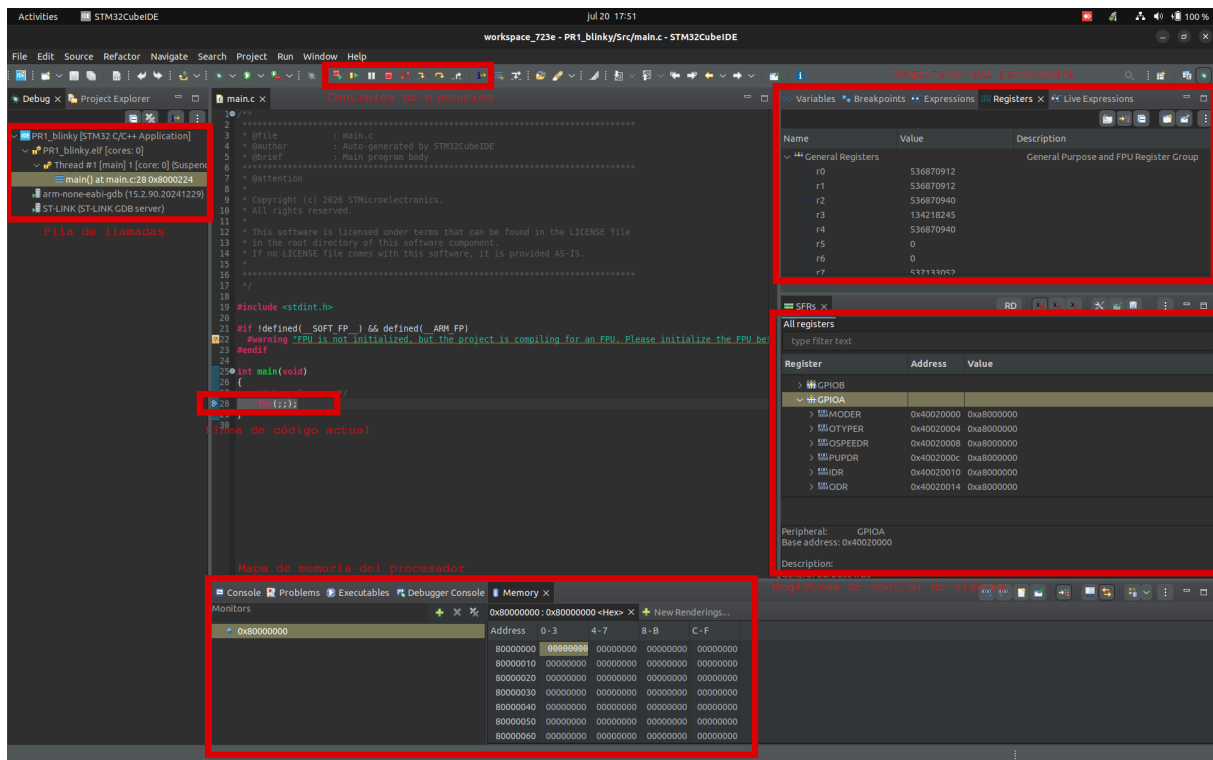


Figura 13: Ventana de depuración personalizada

- Configurar la velocidad a alta en **OSPEEDR**.
- Deshabilitar las resistencias en **PUPDR**.
- Finalmente, se podrá escribir el valor del led (0/1) a través de **ODR**. Otra opción es escribir directamente al registro **BSRR**, que tiene la ventaja de evitar lecturas o escrituras múltiples (aunque para nuestro caso no afecta en nada).

Adicionalmente, tenemos que activar el reloj del periférico GPIOA a través del registro **RCC_AHB1ENR** ([rm0431 5.3.10](#)). La dirección base la podemos consultar en [rm0431 1.6.2](#).

Quizá te quede la pregunta de cómo narices escribo yo en una dirección de memoria concreta... la respuesta son... ¡punteros!

```

1 //Imaginemos que quiero escribir en la dirección 0x40020000
2 //justo en los bits 14-15
3 // (por lo que sea)
4
5 //borro configuración actual (máscara con ceros en las posiciones de
6 //interés)
7 * ((uint32_t *) (0x40020000)) &= ~(0b11 << (7*2));
8 //pongo la configuración que me interesa
9 * ((uint32_t *) (0x40020000)) |= 0b01 << (7*2);

```

Aprovecha también esta función para hacer que parpadee cada cierto tiempo:

```

1 void delay(int loops) {
2     for (int i = 0; i < loops; i++) {
3         __asm("nop");
4     }
5 }

```

2.3.4. Solución

```
1  #include <stdint.h>
2
3  #define GPIOA_BASE_ADDR 0x40020000
4  #define RCC_AHB1ENR 0x40023830
5
6  void delay(int loops) {
7      for (int i = 0; i < loops; i++) {
8          __asm("nop");
9      }
10 }
11
12 int main(void)
13 {
14     //activate clock for GPIOA to work
15     * ((uint32_t *) RCC_AHB1ENR) |= 0x01;    //set bit 0 to 1 to enable
        clock on GPIOA
16
17     * ((uint32_t *) (GPIOA_BASE_ADDR + 0x00)) &= ~(0b11 << 14); //clear
18     * ((uint32_t *) (GPIOA_BASE_ADDR + 0x00)) |= 0b01 << 14; //output
19     * ((uint32_t *) (GPIOA_BASE_ADDR + 0x04)) &= ~(0b0 << 7); //clear
20     * ((uint32_t *) (GPIOA_BASE_ADDR + 0x04)) |= 0b0 << 7; //push-
        pull
21     * ((uint32_t *) (GPIOA_BASE_ADDR + 0x08)) &= ~(0b10 << 14); //clear
22     * ((uint32_t *) (GPIOA_BASE_ADDR + 0x08)) |= 0b10 << 14; //high
        speed
23     * ((uint32_t *) (GPIOA_BASE_ADDR + 0x0C)) &= ~(0b00 << 14); //clear
24     * ((uint32_t *) (GPIOA_BASE_ADDR + 0x0C)) |= 0b00 << 14; //
        disable pullup and down
25
26     /* Loop forever */
27     for(;;) {
28         * ((uint32_t *) (GPIOA_BASE_ADDR + 0x14)) |= 0b1 << 7;    //
            turn on
29         delay(1000000);
30         * ((uint32_t *) (GPIOA_BASE_ADDR + 0x14)) &= ~(0b1 << 7); //
            turn off
31         delay(1000000);
32     }
33 }
```